Information Modeling Using EXPRESS — Modeling Principles

SUMMARY

- Organize the team who will
- Develop the model while
- Applying the principles.

THE TEAM

The modeling team consists of three groups:

1. Technical experts who know the model domain.
2. Modeling experts who help the technical experts develop the model.
3. Reviewers, who are domain experts, to ‘keep the developers honest’

Remember —

> A theory has only the alternative of being right or wrong. A model has a third possibility: it may be right but irrelevant.
>
> — Manfred Eigen in The Physicists Conception of Nature (ed. Jagdish Mehra) Reidel, 1973

PROCESS

- Define the scope
- ‘Find the objects’
- ‘Sketch’ the structure
- Generalisation and specialisation
- Refine the structure
- Eliminate synonyms and homonyms
- Cardinality constraints
- Local constraints
- Global constraints
- Partition

PRINCIPLES

- Scoping
- Readability
- Simple Types
- Implementation Independence
- Partitioning
- Schema Independence
- Nym Principle
- Context Independence
- Constraints
- Invariance
- Redundancy
- Inheritance

Readability

An information model, although if defined via EXPRESS is computer interpretable, should primarily be human understandable.

Modeling constructs should be chosen to aid the reader rather than obfuscate understanding by using complex, intertwined or opaque definitional relationships.

Scoping

A model must have a well defined scope (otherwise how do you know when you are finished?).

Before modeling:

- Define the intended model scope.
- Document the assumptions.

Simple Types

The EXPRESS simple types (i.e Integer, String, etc) carry virtually no semantics.

As far as possible, avoid the use of simple types as entity attributes. Instead, use Types to provide the semantics.

```
ENTITY antique_object;
  produced : date;
```

```
  maker    : name;
DERIVE
  years_old : age := elapsed(produced);
END_ENTITY;

TYPE name = STRING; END_TYPE;
TYPE age = INTEGER; END_TYPE;
```

Implementation Independence

An information model defines the **information** necessary for some purpose; *it does not define the data structuring*.

- Model objects, not syntax.
- Model the 'real world' not a technology-based representation.
- Do not define 'precision' of data types.
- Do not define 'default' data.

Partitioning

- A `SCHEMA` defines a scope and a context.
- A model is often partitioned into a set of schemas (e.g to increase readability, to partition the development work, etc.)
- Minimise the interaction between schemas.
- Within a schema, minimise the constraints on the objects in question (to promote re-usability).

Schema Independence

In EXPRESS, each Schema defines a scope; definition names need only be unique within a Schema.

Attempt to maintain name uniqueness across all schemas in a model (see the Nym Principle). This will assist when restructuring a model, if necessary, by modifying the schema boundaries.

Nym Principle

'If things are the same, then they should have the same name.'

'If things are not the same, then they are different.'

'Different things should have different names.'

In general, 'one name, one meaning, one definition'. Synonyms and homonyms in a model are a fruitful and never-ending source of confusion.

Context Independence

Each entity exists in a context in which it may be used. This may vary from extremely broad to highly specific. An entity definition should be as context independent as possible, yet as context specific as required.

- Only apply the minimum necessary number of constraints.
- Use Subtyping to get more specificity.

Invariance

The meaning of an entity should not be dependent on the values of its attributes. Do not use ‘flags’ to change meanings.

```
ENTITY poor_person_model;
  sex : enumeration_of_male_female;
  ... -- gender related attributes
  ... -- non-gender attributes
END_ENTITY;

ENTITY good_person_model
  SUPERTYPE OF (ONEOF(male, female));
  ... -- non-gender attributes
END_ENTITY;
  -- gender related attributes put into
  -- the relevant subtypes
```

Constraints

An EXPRESS information model is *permissive* (i.e what is not explicitly prohibited is permissable).

Add all necessary constraints — a model is as much about the limitations of objects as it is about the objects themselves.

```
TYPE age = INTEGER;
WHERE
  non_negative : SELF >= 0;
END_TYPE;
```

Constraint ordering

Specify constraints by the following ordered preferences:

1. Model structure

2. Local constraints
3. Global rules

Constraint (global rule)

```
ENTITY male SUBTYPE OF (person);
  wife : OPTIONAL female;
  -- other attributes
END_ENTITY;

ENTITY female SUBTYPE OF (person);
  husband : OPTIONAL male;
  -- other attributes
END_ENTITY;

RULE married FOR (male, female);
  -- check declared husbands
  -- and wives match each other
END_RULE;
```

Constraint (local)

```
ENTITY male SUBTYPE OF (person);
  wife : OPTIONAL female;
  -- other attributes
WHERE
  -- check wife says she is
  -- married to me
END_ENTITY;

ENTITY female SUBTYPE OF (person);
  husband : OPTIONAL male;
  -- other attributes
WHERE
  -- check husband says he is
  -- married to me
END_ENTITY;
```

Constraint (structural)

```
ENTITY male SUBTYPE OF (person);
  -- other attributes
END_ENTITY;
```

```
ENTITY female SUBTYPE OF (person);
  -- other attributes
END_ENTITY;

ENTITY married;
  husband : male;
  wife    : female;
UNIQUE
  no_bigamy    : husband;
  no_polyandry : wife;
END_ENTITY;
```

Redundancy

A model should not contain redundant information; redundancy leads to the possibility of data inconsistencies.

```
ENTITY circle;
  center : point;
  radius : REAL;
DERIVE
  perimeter : REAL := 2.0*PI*radius;
  diameter : REAL := 2.0*radius;
END_ENTITY;
```

Inheritance

A Subtype inherits all the properties of its Supertype.

For readability it may appear desirable to migrate the common properties down to the leaves of the supertype tree. This, however, implies that the common properties are semantically different.

All common properties should be moved as close to the root of the Supertype tree as possible. This demonstrates that they ARE common.

SCHEMA INTERFACING

```
SCHEMA first;
  ENTITY aaa;
    -- attributes
  END_ENTITY;

  ENTITY original;
    attr : NUMBER;
  END_ENTITY;
END_SCHEMA; -- first
```

```
SCHEMA second;
  USE FROM first (aaa AS bbb);
  REFERENCE FROM first (original);

  ENTITY constrained
    SUBTYPE OF (original);
    attr : INTEGER(7);
    WHERE
      positive : attr > 0;
    END_ENTITY;
END_SCHEMA; -- second
```